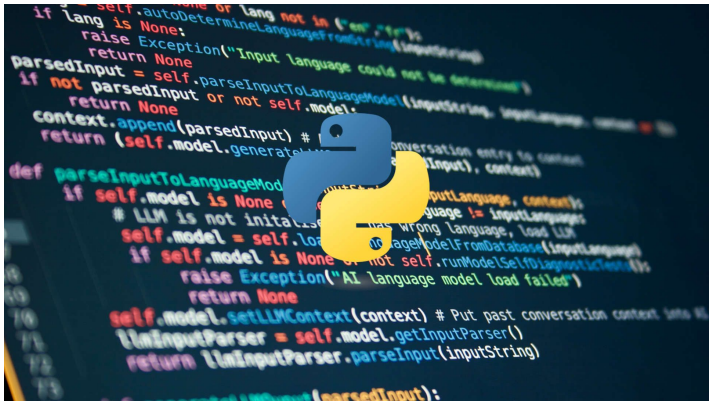
BearAI Workshop #2

Code the Study Assistant
Part 2/3



Today you will:

- Connect Python to an AI model using Gemini API
- Turn Workshop #1 prompt into code
- Build Explain, Quiz, and Flashcard modes
- Make your study assistant interactive
- Customize and test your own study mode



Workshop #1 RECAP

- We built a prompt-based study assistant
- Learned how to structure prompts using Goal + Context + Rules + Output
- Tested assistant and improved the assistant using Virelia notes
- Learned that prompt engineering is about improving iteratively



Get Started

Go to the BearAI website: https://bearai-bu.github.io/#/projects/ai-study-assistant

On the page:

- Find Workshop #2 - Code the Study Assistant
- Open the Student Colab
- In Colab: File -> Save a copy in Drive
- Work only in your own copy
- Follow notebook from top to bottom
- We'll walk through the setup together before you work at your own pace

Part 1 — Connect Python to Gemini

```
!pip install -U google-genai "google-auth==2.49.0"
```

Add this to the end of the pip install command then run the cell.
After, run the API Key cell and paste API key into the text box:

```
import os
from getpass import getpass

os.environ["GEMINI_API_KEY"] = getpass("paste your Gemini API key: ")
```

Finally, run the test cell. You should see: api connected

[Get your free Gemini API key]

Test the connection

Run the test connection cell:

You should see:

```
api connected
```

Get your free Gemini API key



Part 2 — Bring back Workshop #1

In Workshop #1, we manually gave the model:

instructions + notes + user request

Now Python will build that prompt for us.

Run 4. Virelia Notes (leave them as they are), then 5. ask_ai(): it sends a prompt to Gemini and returns the response.

Python Syntax Part 1 - Variables and Functions

Variables: Store data such as integers, floating point numbers, strings (sequences of characters), or objects.

A = 10

string = “Hello World”

Functions: Reusable pieces of code that perform some number of operations

```
def newFunc(p1, p2):
    return f”{p1} and {p2}”
```

- Arguments - What we provide the function
- Parameters - What the function expects us to give it
- Return - What the function will give us

Python — Basic Assignment

```
name = "Leo"          # String
age = 7               # Integer
height = 5.6          # Float
is_cat = True         # Boolean
flaws = None          # None type
```

Python — Parallel & Chained Assignments

```
x, y = 10, 20         # Assign multiple values
a = b = c = 0         # Give same value to multiple variables
```

Python Syntax Part 2 - Dictionaries

A **dictionary** is a store of **key-value pairs** which allow us to access individual elements by using the key.

dict = {key: value}

newDict = **{**

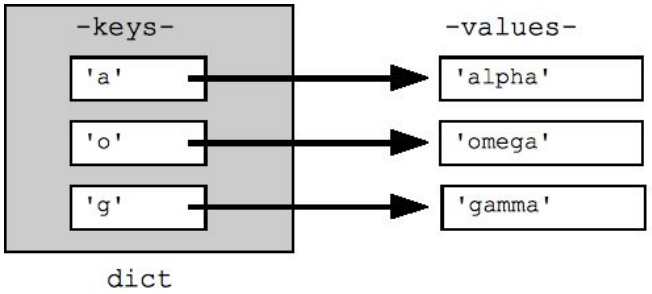
1: “Hi”,

2: “Hello”,

3: “Salutations”

}

newDict[1] returns “Hi”



Functions

Functions are named blocks of code, designed to do one specific job. Information passed to a function is called an argument, and information received by a function is called a parameter.

A simple function

```
def greet_user():
    """Display a simple greeting."""
    print("Hello!")

greet_user()
```

Passing an argument

```
def greet_user(username):
    """Display a personalized greeting."""
    print(f"Hello, {username}!")

greet_user('jesse')
```

Default values for parameters

```
def make_pizza(topping='pineapple'):
    """Make a single-topping pizza."""
    print(f"Have a {topping} pizza!")

make_pizza()
make_pizza('mushroom')
```

Returning a value

```
def add_numbers(x, y):
    """Add two numbers and return the sum."""
    return x + y

sum = add_numbers(3, 5)
print(sum)
```

Variables and Strings

Variables are used to assign labels to values. A string is a series of characters, surrounded by single or double quotes. Python's f-strings allow you to use variables inside strings to build dynamic messages.

Hello world

```
print("Hello world!")
```

Hello world with a variable

```
msg = "Hello world!"
print(msg)
```

f-strings (using variables in strings)

```
first_name = 'albert'
last_name = 'einstein'
full_name = f"{first_name} {last_name}"
print(full_name)
```

Defining a dictionary

Use curly braces to define a dictionary. Use colons to connect keys and values, and use commas to separate individual key-value pairs.

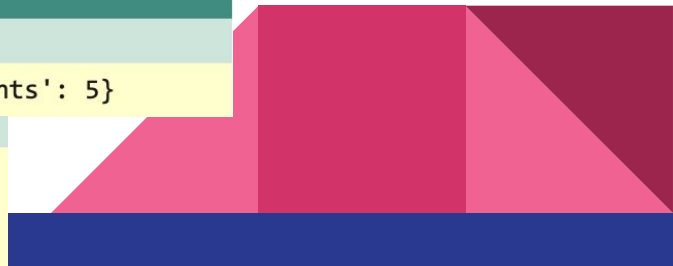
Making a dictionary

```
alien_0 = {'color': 'green', 'points': 5}
```

Getting the value associated with a key

```
alien_0 = {'color': 'green', 'points': 5}

print(alien_0['color'])
print(alien_0['points'])
```



Use AI to Support Learning

Good uses

- Ask for explanations
- Practice concepts
- Generate study questions
- Get feedback on understanding

Remember

- Follow your professor/course AI policy
- Verify AI-generated info

Workshop #3 Preview

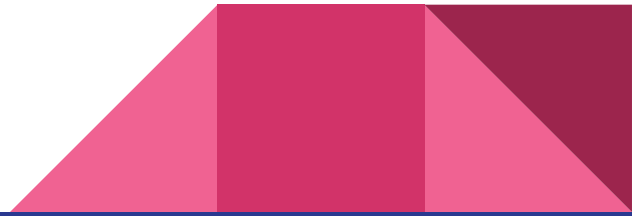
Today, our program sends all of the notes to Gemini every time

What if we had an entire semester of material?

Workshop #3 will make our application find relevant notes first, then send only those to the AI model

RAG = Retrieval-Augmented Generation

Save your completed Colab, keep access to your Gemini API key, bring laptop + Google account









Stay connected with BearAI! Website: bearai-bu.github.io